

UNIVERSITÀ DEGLI STUDI DI UDINE

DIPARTIMENTO DI SCIENZE
MATEMATICHE, INFORMATICHE E
FISICHE

ANNO ACCADEMICO 2023/2024



Algoritmi e Strutture dati Progetto di Laboratorio

Autori

Aleksic Aleksa, 161711
Antonello Samuele, 162640
Coppo Cristian, 161833

aleksic.aleksa@spes.uniud.it
antonello.samuele@spes.uniud.it
coppo.cristian@spes.uniud.it

Contents

1	Introduzione	2
1.1	Descrizione generale del problema	2
2	Analisi dei tempi	2
2.1	Descrizione	2
2.2	Lunghezze dei vettori	3
2.3	Generazione dei vettori	3
2.4	Resolution	4
2.5	Tempi medi di esecuzione	5
2.6	Benchmark	6
3	Quick Select	7
3.1	Descrizione	7
3.2	Partition	7
3.3	Codice	8
3.4	Complessità	8
4	Heap Select	9
4.1	Descrizione	9
4.2	La struttura dati Heap	9
4.3	Codice	10
4.4	Heap select con variante	11
4.5	Codice con variante	11
4.6	Complessità	12
5	Medians of Medians Select	13
5.1	Descrizione	13
5.2	Codice	14
5.3	Complessità	14
6	Analisi comparativa	15
6.1	Grafici in scala lineare	15
6.2	Grafici in scala logaritmica	16
6.3	Grafici comparativi	17
6.4	Osservazioni	18

1 Introduzione

1.1 Descrizione generale del problema

L'obiettivo dell'esercitazione è quello di implementare tre algoritmi di selezione per poi condurre un'analisi ed una stima dei tempi medi di esecuzione di tali algoritmi in funzione della dimensione dell'input.

Il problema della selezione si concentra sul trovare la posizione del k -esimo elemento più piccolo in un vettore ordinato di interi.

Per tale scopo è quindi necessario implementare gli algoritmi in un qualsiasi linguaggio di programmazione e in seguito valutarne la complessità computazionale in termini di spazio e di tempo.

Per lo svolgimento di questa esperienza è stato scelto il linguaggio Python ed i tre algoritmi di selezione la cui implementazione è stata richiesta sono Quick select, Heap select e Medians-of-medians select.

2 Analisi dei tempi

2.1 Descrizione

Per quanto concerne l'analisi della complessità temporale degli algoritmi implementati è richiesta la realizzazione di un programma per la misurazione dei tempi medi di esecuzione di tali algoritmi, al variare della lunghezza m del vettore e del parametro k forniti in input.

I tre algoritmi vanno valutati per vettori di lunghezza n compresa tra 100 e 100000, inoltre per ogni lunghezza di tale intervallo è necessario generare almeno 100 campioni. Le possibili lunghezze devono, inoltre, seguire una serie geometrica.

Questo viene fatto allo scopo di ottenere una misurazione di un tempo medio per quel determinato valore di n .

2.2 Lunghezze dei vettori

Per generare le lunghezze dei vettori di test è stato utilizzato un ciclo for con un indice i , il quale è stato fatto variare da 0 a 99. La lunghezza del vettore è stata definita come funzione esponenziale di i :

$$n_i = \lfloor A * B^i \rfloor$$

Dove A e B sono costanti in virgola mobile calcolate opportunamente in modo da ottenere

- $n_i = 100$ per $i = 0$
- $n_i = 100000$ per $i = 99$

Dai calcoli effettuati otteniamo, quindi, i valori da assegnare ad A e B:

- $A = 100$
- $B = 1000^{\frac{1}{99}}$

Le lunghezze generate in tale modo sono state poi memorizzate in un vettore.

2.3 Generazione dei vettori

Ogni test effettuato su una lunghezza n deve generare in modo pseudo-casuale un vettore di n interi, in un intervallo a scelta. Nel nostro caso questi interi sono stati fatti variare da 1 a 1000000.

Per ogni test deve essere generato, sempre in modo pseudo-casuale, anche un indice k , il quale verrà poi passato in input agli algoritmi di selezione. Questo parametro deve quindi essere compreso in un intervallo $\{1, \dots, n\}$. Ecco il codice per realizzare tutto ciò:

```
1  def init_array(n):
2      #genera un array di numeri interi casuali di dimensione n
3      #in un intervallo 1 - 1000000
4      return [random.randint(1, 1000000) for i in range(n)]
5
6  def init_index(n):
7      #genera un indice k casuale tra 0 e n-1
8      return random.randint(0, n - 1)
```

2.4 Resolution

Un aspetto fondamentale dell'esercitazione è la stima dei tempi medi di esecuzione degli algoritmi di selezione su un vettore di interi. Essa deve, inoltre, garantire un errore relativo massimo pari a 0.001.

Per tutte le misurazioni del tempo trascorso è stato utilizzato un clock di sistema monotono.

Le misurazioni di tempo sono state effettuate utilizzando la funzione `perf_counter()` del modulo `time` del linguaggio Python.

È stata implementata, quindi, la funzione `resolution()`, la quale fornisce come valore di ritorno una stima della risoluzione del clock di sistema, sfruttando un ciclo `while` per calcolare l'intervallo minimo di tempo misurabile.

```
1 def resolution():
2     start = time.perf_counter()
3     while time.perf_counter() == start:
4         pass
5     stop = time.perf_counter()
6     return stop - start
```

È stato poi calcolato il tempo minimo misurabile in funzione della risoluzione stimata R e dell'errore relativo massimo ammissibile $E = 0.001$:

$$T_{min} = R * (\frac{1}{E} + 1)$$

2.5 Tempi medi di esecuzione

Per stimare il tempo medio di esecuzione di un algoritmo su istanze dell'input di dimensione n è stato utilizzato un ciclo while, andando ad iterare l'esecuzione dell'algoritmo su un input di dimensione n , generato in modo pseudo-casuale, come spiegato nei paragrafi precedenti, e misurando complessivamente un intervallo di tempo superiore a T_{min} .

Questa misurazione deve essere fatta senza interrompere il clock, ovvero calcolando l'intero intervallo di tempo trascorso dall'inizio dell'iterazione a quando il tempo misurato risulta maggiore di T_{min} .

Il tempo medio di una singola istanza di input, successivamente, si ottiene dal rapporto tra il tempo totale misurato e il numero di iterazioni dell'algoritmo che sono state eseguite.

Le operazioni descritte sopra sono state realizzate tramite l'implementazione della procedura *measure()*, riportata a seguire.

```
1 def measure(n, min_time, select_algorithm):
2     count = 0;
3     start_time = time.perf_counter()
4     while True:
5         vector = init_array(n)
6         index = init_index(n)
7         select_algorithm(vector, 0, n-1, index)
8         count += 1
9         end_time = time.perf_counter()
10        if end_time - start_time >= min_time:
11            break
12        average_time = (end_time - start_time) / count
13    return average_time
```

2.6 Benchmark

A questo punto è stata realizzata la procedura principale che va ad effettuare il benchmark vero e proprio.

La procedura *Benchmark* riceve in input uno dei tre algoritmo di selezione implementati e il nome del suddetto algoritmo.

Il vettore *mean_times* è stato inizializzato per contenere i tempi medi di esecuzione per ogni lunghezza di test generata.

L'idea è quella di eseguire 100 volte la procedura *measure()* per ogni lunghezza contenuta nel vettore delle lunghezze precedentemente generato.

Alla base del funzionamento dell'algoritmo, troviamo un ciclo for, il quale ad ogni iterazione eseguita in base alla lunghezza *n* del vettore di test alloca un vettore inizialmente vuoto chiamato *time_vector*.

Successivamente, possiamo notare un ciclo for annidato, che ha lo scopo di eseguire per un totale di 100 iterazioni la procedura *measure()*, per poi allocare i risultati di quest'ultima nel vettore *time_vector*.

Al termine di questo ciclo viene, quindi, calcolato il tempo medio di esecuzione in relazione alla lunghezza *n*, ottenuto dalla media dei tempi contenuti nel vettore *time_vector*.

Il valore così calcolato viene poi inserito nel vettore *mean_times*, il quale viene poi utilizzato per realizzare il grafico dei tempi medi di esecuzione in funzione della lunghezza del vettore in input, dell'indice *k* generato in modo pseudo-casuale e dell'algoritmo di selezione utilizzato.

A seguire è riportato il codice della procedura *benchmark()*

```
1 def benchmark(algoritmo,name):
2     mean_times = []
3     for n in range_test:
4         time_vector = []
5         for x in range(100):
6             t = measure(n, T_min, algoritmo)
7             time_vector.append(t)
8         tempo_medio = sum(time_vector) / len(time_vector)
9         mean_times.append(tempo_medio)
10        print("Tempo medio di esecuzione per n =",
11              n, ">", tempo_medio, "secondi")
12        plt.plot(range_test,mean_times,"o",label = name)
```

3 Quick Select

3.1 Descrizione

L'algoritmo di selezione Quick select è una variante dell'algoritmo di ordinamento "quick sort", nel quale ogni chiamata ricorsiva su un intervallo $[i, j]$ del vettore che viene fornito in input termina restituendo:

- il k -esimo elemento più piccolo del vettore se $k - 1 \in [i, j]$
- un valore indefinito se $k - 1 \notin [i, j]$

Nel secondo caso, inoltre, una chiamata sull'intervallo $[i, j]$ può terminare in tempo costante $\Theta(1)$ senza eseguire la procedura partition.

3.2 Partition

La procedura Partition, che viene richiamata nel primo caso, prende in input un vettore di interi e due indici corrispondenti alle posizioni del primo e dell'ultimo elemento del vettore.

Essa prende come perno l'ultimo elemento del vettore e ha lo scopo di riarrangiare gli elementi che esso contiene in modo tale che:

- a sinistra del perno siano presenti tutti gli elementi minori di esso
- a destra del perno siano presenti tutti gli elementi maggiori di esso

Al termine del ciclo for la procedura ritorna la posizione del pivot dopo le modifiche fatte al vettore.

Partition è un algoritmo in-place e ha complessità computazionale $\Theta(n)$, con $n = q - p + 1$ che è la lunghezza del vettore.

```
1  def Partition(A,p,q):
2      x = A[q]
3      i = p-1
4      j=p
5      for j in range (p,q):
6          if A[j] <= x:
7              i += 1
8              A[i],A[j] = A[j],A[i]
9      A[i+1],A[q] = A[q],A[i+1]
10     return i+1
```

3.3 Codice

```
1 def QuickSelect(A,p,q,k):
2     if k<p:
3         return None
4     if p==q:
5         return A[p]
6     else:
7         r = Partition(A,p,q)
8         if k==r:
9             return A[r]
10        elif k<r:
11            return QuickSelect(A,p,r-1,k)
12        else:
13            return QuickSelect(A,r,q,k)
14
```

3.4 Complessità

L'algoritmo di selezione Quick select ha una complessità temporale asintotica $\mathcal{O}(n)$ nel caso medio e $\Theta(n^2)$ nel caso peggiore. Possiamo notare come la complessità di Quick select sia più bassa di quella dell'algoritmo di ordinamento "quick sort", il quale utilizza una logica molto simile.

La ridotta complessità è data dal fatto che, dopo aver partizionato il vettore attorno al perno, Quick select utilizza la ricorsione solamente sulla metà di vettore a cui si è interessati e non su entrambe come nell'algoritmo di ordinamento. Ciò permette di ottenere la seguente equazione ricorsiva di complessità nel caso medio:

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(n)$$

Da cui si ottiene una complessità $\mathcal{O}(n)$

Il caso peggiore, invece, si ottiene quando, per esempio, si cerca la posizione dell'ultimo elemento del vettore e per fare ciò vengono richieste n chiamate ricorsive.

Per quanto riguarda la complessità in termini di spazio, l'algoritmo non è in-place, in quanto utilizza lo stack delle chiamate ricorsive.

4 Heap Select

4.1 Descrizione

L'algoritmo di selezione Heap select è realizzato sfruttando due min-heap, che sono state denominate H1 e H2:

- H1 viene costruita in tempo lineare a partire dal vettore fornito in input
- H2 è costituita inizialmente da un solo nodo, corrispondente alla radice di H1

Esso riceve in input un vettore di interi, due indici corrispondenti alle posizioni di inizio e fine del vettore e un indice k per il quale si vuole conoscere che elemento finirebbe in tale posizione se il vettore venisse ordinato.

Dopo l'inizializzazione delle due heap come descritto sopra, l'algoritmo presenta un ciclo for che scandisce la heap sino alla posizione del vettore che si sta cercando.

All' i -esima iterazione del ciclo for, per i che va da 1 a $k - 1$, l'algoritmo estrae la radice di H2, la quale corrisponde ad un nodo x_i nella heap H1, e reinserisce in H2 i nodi successori di x_i , i quali sono il suo figlio destro ed il suo figlio sinistro in H1.

Dopo $k - 1$ iterazioni, la radice della heap H2 corrisponderà al k -esimo elemento più piccolo del vettore fornito in input.

4.2 La struttura dati Heap

Una heap è un albero binario quasi completo nel quale ogni nodo ha chiave intera:

- $\geq$ a quella dei suoi figli se è una max-heap
- $\leq$ a quella dei suoi figli se è una min-heap

Una heap può essere memorizzata tramite un vettore sovradimensionato, nel quale considerando l'elemento k :

- il suo figlio sinistro si trova in posizione $2k$
- il suo figlio destro si trova in posizione $2k + 1$

4.3 Codice

```
1 def heap_select(a,p,q,k):
2     H1 = MinHeap()
3     H2 = MinHeap()
4     low = 0
5     high = k
6     H1.buildHeap(a)
7     H2.insert(H1.getRoot())
8     for i in range(low,high):
9         extracted = H2.extract()
10        left = H2.left(extracted[1])
11        right = H2.right(extracted[1])
12        if left < H1.length():
13            H2.insert(H1.heap[left])
14        if right < H1.length():
15            H2.insert(H1.heap[right])
16    return H2.getRoot()
```

4.4 Heap select con variante

Per rendere l'algoritmo efficiente sia per k piccoli che per k grandi è stata implementata una versione di Heap select che utilizza opportunamente min-heap o max-heap valutando la posizione k all'interno del vettore.

- se k si trova nella parte destra del vettore, vengono utilizzate le max-heap
- se k si trova nella parte sinistra del vettore, vengono utilizzate le min-heap

L'idea alla base di ciò è che se il k -esimo elemento più piccolo è vicino alla fine del vettore, utilizzando le min-heap farei più estrazioni rispetto a se risolvessi il problema con le max-heap e viceversa.

4.5 Codice con variante

```
1 def heap_select(a,p,q,k):
2     if h>(len(a)//2):
3         H1 = MaxHeap()
4         H2 = MaxHeap()
5         low = k
6         high = len(a)-1
7     else:
8         H1 = MinHeap()
9         H2 = MinHeap()
10        low = 0
11        high = k
12    H1.buildHeap(a)
13    H2.insert(H1.getRoot())
14    for i in range(low,high):
15        extracted = H2.extract()
16        left = H2.left(extracted[1])
17        right = H2.right(extracted[1])
18        if left < H1.length():
19            H2.insert(H1.heap[left])
20        if right < H1.length():
21            H2.insert(H1.heap[right])
22    return H2.getRoot()
```

4.6 Complessità

L'algoritmo di selezione Heap select ha complessità temporale $\mathcal{O}(n + k \log k)$ sia per quanti riguarda il caso medio, che per quanto concerne il caso peggiore. La complessità rimane invariata anche per l'algoritmo con la variante.

Tale costo computazionale è dato da:

- un costo $\mathcal{O}(n)$ per costruire la heap partendo da un vettore di n elementi
- un costo $\mathcal{O}(k \log k)$, dato dal numero di operazioni effettuate per trovare il k -esimo elemento, ognuna delle quali ha un costo $\mathcal{O}(\log k)$.

5 Medians of Medians Select

5.1 Descrizione

L'ultimo dei tre algoritmi di selezione che è stato implementato durante la realizzazione di questo progetto è Medians-of-medians select.

Quest'ultimo è basato sulla suddivisione del vettore fornito in input in blocchi di dimensione limitata e sul calcolo della mediana delle mediane.

Nello specifico, l'algoritmo esegue le seguenti operazioni:

- divisione del vettore in blocchi di 5 elementi, escluso l'ultimo blocco che potrà contenerne una quantità inferiore
- ordinamento e calcolo della mediana di ciascun blocco
- calcolo della mediana M delle mediane dei blocchi, attraverso una chiamata ricorsiva allo stesso algoritmo
- partizionamento del vettore attorno alla mediana M , attraverso una variante della procedura *partition* dell'algoritmo *quick sort*
- chiamata ricorsiva nella parte di vettore che sta a sinistra o a destra della mediana M , in funzione del valore k fornito in input.

La versione dell'algoritmo che è stata realizzata da noi è considerabile "quasi in place", dato che l'unico spazio aggiuntivo utilizzato è quello dedicato alla pila che gestisce le chiamate ricorsive.

Per rendere l'algoritmo Medians-of-medians select "quasi in place" è stato sfruttato lo spazio del vettore fornito in input per memorizzare i vari mediani. Essi, infatti, una volta calcolati, vengono spostati nelle prime posizioni del vettore.

L'algoritmo di ordinamento che è stato utilizzato è *insertion sort*, modificato opportunamente per poter ordinare porzioni di vettore comprese fra due indici passati in input.

La procedura *partition*, utilizzata per partizionare il vettore attorno alla mediana delle mediane non ha subito grandi modifiche rispetto alla versione implementata in *quick select*.

Questo perché è stato preferito spostare direttamente il mediano dei mediani nell'ultima posizione del vettore in modo tale che il *partition* classico lo prendesse come perno.

È stato scelto di dividere l'algoritmo in due procedure per separare il calcolo del mediano dei mediani dall'algoritmo di selezione vero e proprio che ritorna in output il k -esimo elemento più piccolo del vettore fornito in input.

5.2 Codice

```
1 def momCalcolo(arr,low,high,k):
2     j = low
3     for i in range(low,high+1,5):
4         local_min = min((i+4),high)
5         insertionSort(arr,i,local_min)
6         arr[j],arr[(i+local_min)//2] =
7             arr[(i+local_min)//2],arr[j]
8         j += 1
9     return medianOfmedians(arr,low,j-1,(low + j-1)//2)
```

```
1 def medianOfmedians(arr,low,high,k):
2     if high == k and k ==low:
3         return k
4     m = momCalcolo(arr,low,high,k)
5     arr[high], arr[m] = arr[m], arr[high]
6     pi = Partition(arr,low,high)
7     if k == pi:
8         return pi
9     elif k < pi:
10        return medianOfmedians(arr,low,pi-1,k)
11    else:
12        return medianOfmedians(arr,pi+1,high,k)
```

5.3 Complessità

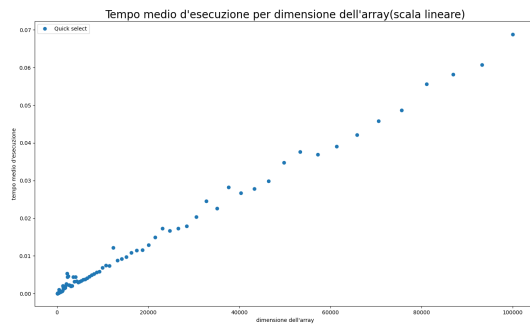
L'algoritmo di selezione medians-of-medians select combina il partizionamento del vettore e la ricerca del mediano dei mediani, operazioni che vengono svolte entrambe in tempo lineare.

L'algoritmo in analisi, ha quindi una complessità temporale, nel caso pessimo, pari a $\mathcal{O}(n)$.

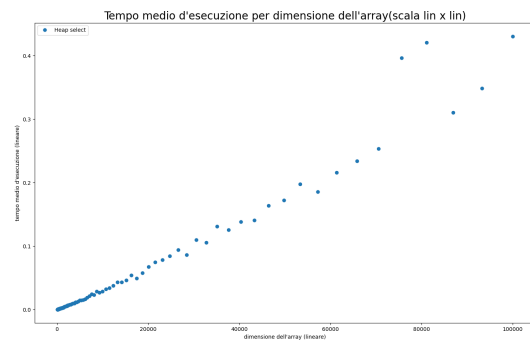
Per quanto riguarda la complessità spaziale, invece, l'algoritmo, nella versione da noi realizzata, è considerabile "quasi in place" e quindi con costo in termini di spazio approssimabile a $\Theta(1)$.

6 Analisi comparativa

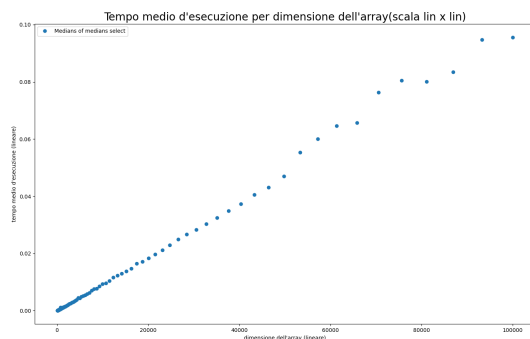
6.1 Grafici in scala lineare



Quick Select

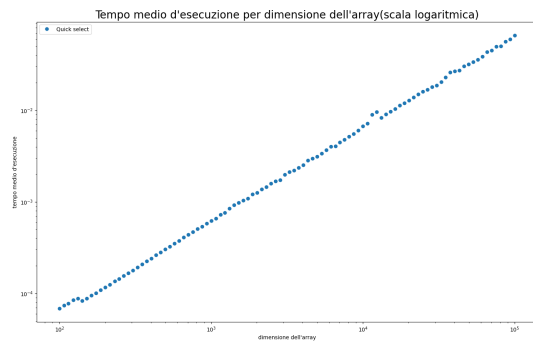


Heap Select

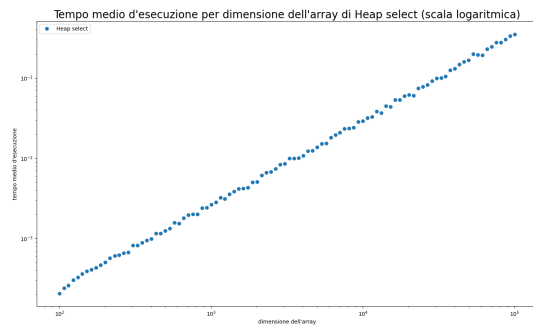


Medians-of-medians
Select

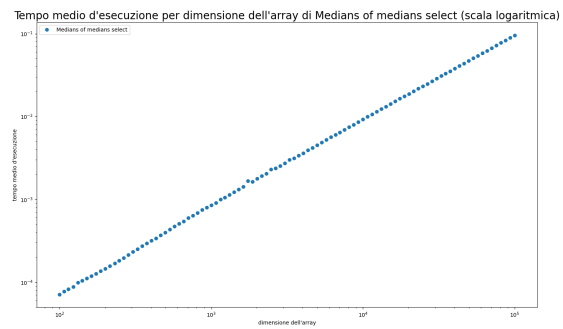
6.2 Grafici in scala logaritmica



Quick Select



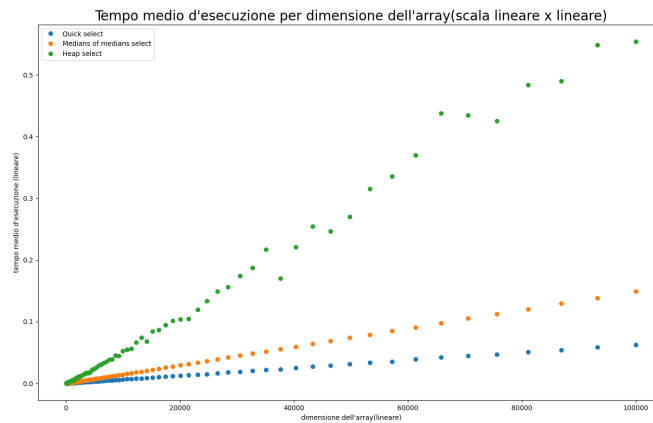
Heap Select



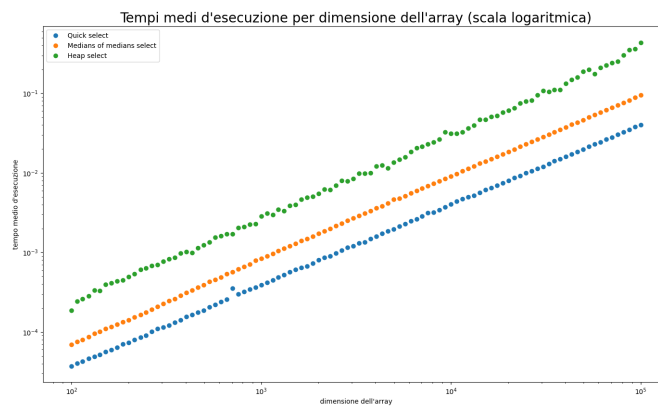
Medians-of-medians
Select

6.3 Grafici comparativi

Scala lineare



Scala logaritmica



6.4 Osservazioni

L'algoritmo di selezione *Quick select* ha complessità computazionale, in termini di tempo, pari a $\mathcal{O}(n^2)$ nel caso peggiore e $\Theta(n)$ nel caso medio. Per quanto riguarda la complessità spaziale, invece, esso non è in-place in quanto utilizza lo stack per le chiamate ricorsive.

L'algoritmo *Heap select* ha complessità computazionale temporale $\mathcal{O}(n + k \log k)$ sia nel caso medio che nel caso pessimo, con k che è il k -esimo elemento più piccolo del vettore fornito in input. Data l'implementazione delle versione dell'algoritmo che sfrutta sia max-heap che min-heap, la componente $k \log k$ raggiunge il valore massimo quando l'indice k è situato a metà del vettore. Per quanto riguarda lo spazio utilizzato, l'algoritmo non è assolutamente in-place, in quanto utilizza una heap ausiliaria.

L'algoritmo *Medians-of-medians select* ha complessità computazionale temporale $\mathcal{O}(n)$ sia nel caso medio che in quello pessimo. In termini di spazio, invece, nella versione da noi realizzata si può dire che l'algoritmo è "quasi in-place". Questo poiché l'unico spazio ausiliari allocato è quello necessario per lo stack delle chiamate ricorsive.

Costo computazionale in termini di tempo

	Costo computazionale	
	Caso medio	Caso pessimo
Quick select	$\mathcal{O}(n)$	$\Theta(n^2)$
Heap select	$\mathcal{O}(n + k \log k)$	$\mathcal{O}(n + k \log k)$
Medians-of-medians select	$\mathcal{O}(n)$	$\mathcal{O}(n)$